Radix Sort and Ternary Search

CSE 4403

Algorithm

Antara Arifa Mullick

ID: 210042132

4th Semester (Summer)

AY 2022-2023

B.Sc. In Software Engineering

Dept. of Computer Science and Engineering



LET’S HEAR SOME STORIES

A librarian was tasked with organizing thousands of books not just by their titles but by their unique catalog numbers. These catalog numbers were long and consisted of multiple digits. The librarian wanted a quick and efficient way to sort the books without comparing each digit individually in a time-consuming manner. She remembered an old technique called Radix Sort, which sorts numbers digit by digit, starting from the least significant digit to the most significant one.



LET’S HEAR SOME STORIES



In a vast desert, an explorer was looking for a hidden oasis. The map he had was divided into sections, but he knew the oasis could only be found within a specific range. To minimize his search effort, he decided to use a method where he would divide his search area into three parts and determine which part had the oasis based on some clues he received. This method, known as Ternary Search, helped him efficiently narrow down the location of the oasis.

NAÏVE APPROACH

Story-1

- Sorting numbers would be a simple comparison-based sort, such as **Bubble Sort.**

Story-2

- Finding an element in a sorted array would be a simple **linear search.**

RADIX SORT

- Radix sort is a non-comparative integer sorting algorithm that sorts numbers by processing individual digits.
- The algorithm works by sorting the numbers in multiple passes, where each pass sorts the numbers based on a specific digit or group of digits.

The **librarian's problem** of sorting many books efficiently can be tackled using **Radix Sort**.

RADIX SORT

- Radix Sort works by processing each digit of the numbers.
- Starting from the least significant digit to the most significant digit.
- Uses a stable counting sort as a subroutine.

Pseudo-code:

```
function countingSort(arr, exp):
    n = length(arr)
    output = array of size n
    count = array of size 10 initialized to 0

    for i = 0 to n-1:
        index = (arr[i] / exp) % 10
        count[index]++

    for i = 1 to 9:
        count[i] += count[i-1]

    for i = n-1 downto 0:
        index = (arr[i] / exp) % 10
        output[count[index] - 1] = arr[i]
        count[index]--

    for i = 0 to n-1:
        arr[i] = output[i]

function radixSort(arr):
    maxNumber = getMax(arr)
    exp = 1
    while maxNumber / exp > 1:
        countingSort(arr, exp)
        exp *= 10
```

RADIX SORT: SIMULATION

Initial List	: 170, 45, 75, 90, 802, 24, 2, 66
Sorting by unit's place	: 170, 90, 802, 2, 24, 45, 75, 66
Sorting by ten's place	: 802, 2, 24, 45, 66, 170, 75, 90
Sorting by hundred's place	: 2, 24, 45, 66, 75, 90, 170, 802

RADIX SORT: COMPLEXITY

Time Complexity

- **Best Case:** $\Omega(d(n+k))$
- **Average Case:** $\theta(d(n+k))$
- **Worst Case:** $O(d(n+k))$

Space Complexity

- **All Case:** $O(n+k)$

TERNARY SEARCH

- Ternary search is a **Divide-and-Conquer algorithm** used to find the maximum or minimum value of a unimodal function or to locate a specific element in a sorted array.
- The algorithm operates by dividing the search interval into three parts, as opposed to binary search, which divides the interval into two parts.

The **explorer's problem** of locating the oasis can be tackled using **Ternary Search.**

TERNARY SEARCH

- Ternary Search works by dividing the search interval into three parts and comparing the function values at the two points that divide the interval.
- It then reduces the search space to the part that contains the optimum

Pseudo-code:

```
function ternarySearch(l, r, f,
isMaximizing):
    while r - l > epsilon:
        mid1 = l + (r - l) / 3
        mid2 = r - (r - l) / 3
        if isMaximizing:
            if f(mid1) < f(mid2):
                l = mid1
            else:
                r = mid2
        else:
            if f(mid1) > f(mid2):
                l = mid1
            else:
                r = mid2
    return (l + r) / 2
```

TERNARY SEARCH: SIMULATION

Consider a unimodal function $f(x) = -1(x-2)^2$ on the interval [0, 4].

Initial interval: [0, 4]

```
mid1 = 0 + (4 - 0) / 3 = 1.33
mid2 = 4 - (4 - 0) / 3 = 2.67
f(mid1) < f(mid2), so update
interval to [1.33, 4]
```

Next interval: [1.33, 4]

```
mid1 = 1.33 + (4 - 1.33) / 3 = 2.22
mid2 = 4 - (4 - 1.33) / 3 = 3.11
f(mid1) < f(mid2), so update
interval to [2.22, 4]
```

Next interval: [2.22, 4]

```
mid1 = 2.22 + (4 - 2.22) / 3 = 2.81
mid2 = 4 - (4 - 2.22) / 3 = 3.41
f(mid1) < f(mid2), so update interval
to [2.81, 4]
```

Continue until interval is sufficiently small:

```
Result: x ≈ 2 (where the maximum value
is reached)
```

TERNARY SEARCH: COMPLEXITY

Time Complexity

- **Best Case:** $\Omega(1)$
- **Average Case:** $\theta(\log_3 n)$
- **Worst Case:** $O(\log_3 n)$

Space Complexity

- **Iterative:** $O(1)$
- **Recursive:** $O(\log_3 n)$

ONLINE PROBLEMS

Radix Sort

- Sort-list
- Wiggle-sort-ii
- Sort-by

Ternary Search

- Search-insert-position
- Word-search
- Word-search-ii

REFERENCES

1. "Introduction to Algorithms" by Thomas H. Cormen.
2. Geeksforgeeks: Radix-sort
3. Geeksforgeeks: Ternary Search

Thank You!